

Week 7 — Admission, Scheduling and Routing

Colin Duggan · simulated serving system, no GPU. Every figure is read from results/; seed 7 throughout, DRR in Part 3 so routing is the only variable. Modules gateway/admit.py, gateway/sched.py, router.py, serve.py, each with a .md companion. Plots plots/soak.png (T1) and plots/t3_stale.png (T3) are referenced by path, not embedded, to hold the page limit.

Part 1 — which admission rule embodies which production pattern

Pattern	Rule in gateway/admit.py	What it does
DALL·E request cancellation	Gate 3 — <code>_check_queue_wait</code> → 503	Refuses when <code>queue_length × tft_p50</code> exceeds half the request's deadline, so work that cannot finish inside the budget never enters and holds no memory. Half rather than all, because the estimate covers queue wait only — prefill and decode still follow.
Anthropic late-capacity behaviour	Gates 4 and 5 — <code>_check_kv_pressure</code> , <code>_check_kv_capacity</code> ; then <code>sched._reclaim_blocks</code>	Degrades what is <i>accepted</i> as memory tightens rather than failing everything equally. Below 8% free KV a cold prefix is refused but a resident one is still admitted: under pressure the cheap request is the one worth taking. Gate 5 catches a request big enough to exceed real block capacity while usage still reads under 92%.
Cloudflare overload protection	Gate 6 — <code>_check_tail_latency</code> → 503	When <code>p99 > 4 × p50</code> <i>and</i> the queue is growing, sheds priority ≥ 10 and keeps priority < 10 — shed by class, not at random, so background work absorbs the loss. Both conditions are required; a bad p99 alone may be a blip.

Correction on the DALL·E figure. The timeout had been cut from five minutes to one before the incident. gateway/admit.md still says five minutes; that is wrong, and the corrected number is the better argument. Gate 3 measures against `req.deadline_s`, a per-request input rather than a constant in the gate, so cutting the product's deadline from 300 s to 60 s tightens admission on its own. A gate holding a hard-coded 300 would have kept admitting doomed work from the day the product changed its mind, silently.

Part 2 — which scheduler wastes the most decode work

traces/mixed.jsonl — 444 requests over 60 s at 7 req/s (~1.14× the derived 6.15 req/s capacity), one worker, identical replay for all three policies. Every rejection was a 503; tenant limits were deliberately generous so quota refusals could not mask the capacity rules.

Metric	FCFS	Priority	DRR
completed req/s	3.53	5.70	5.77
interactive p99 TTFT	10.99 s	0.24 s	0.38 s
batch p99 TTFT	11.99 s	6.33 s	2.70 s
preempts	114	17	10
wasted decode tokens	158	125	524
rejected (503)	195	70	69
<i>derived</i> : tokens wasted per preempt	1.4	7.4	52.4

DRR, by more than three times, and it does it while preempting the fewest times. The separation is not in the number of evictions but in the cost of each: 52.4 tokens per preempt against priority's 7.4 and FCFS's 1.4. The cause is DRR's own guarantee. Because every tenant gets a steady share of the prefill slot, a batch request finishes prefill and then decodes continuously, accumulating hundreds of tokens before memory pressure evicts it — and preemption here is recompute, not swap, so all of them are thrown away. FCFS is congested enough that its victims have barely started generating: blocks from a finished prefill, almost no output to lose. Priority starves batch at the *prefill* stage, so it never reaches a state worth losing either. Fairness and wasted work pull against each other: the policy that refuses to starve anyone is the one that guarantees its victims have something valuable to lose.

One trace, one seed, though the ordering held at 8 req/s too (1.4 / 8.8 / 20.8 tokens per preempt). Wasted *prefill* is not counted, per the spec, so true waste is larger than the metric reports.

Part 3, T1 — does power-of-two-choices balance better than random?

traces/t1_unique_prefix.jsonl — 1,296 requests, 90 s at 14 req/s, every prefix distinct, DRR, 2 workers.

Strategy	p99 TTFT	routing imbalance	completed/s	shed
random	9.86 s	0.031	13.03	5.6%
least_loaded	1.17 s	0.051	13.98	0%
p2c	1.17 s	0.051	13.98	0%
prefix_then_load	1.17 s	0.051	13.98	0%

Yes against random: 8.4× better p99 (ratio 0.118) and no shedding against 5.6%. But the three load-reading rows are byte-identical, and that is not a coincidence to explain away. **On two workers, power-of-two-choices is least-loaded** — `rng.sample(candidates, 2)` on a two-element list is the full scan with a shuffle in front of it. Any p2c-vs-least-loaded claim at this fleet size measures the tie-break order and the RNG. Re-run at 4 and 8 workers, where they genuinely differ:

Fleet	random p99	least_loaded p99	p2c p99	least_loaded imbalance	p2c imbalance
4 workers	3.03 s	1.24 s	1.26 s	0.015	0.042
8 workers	2.13 s	0.66 s	0.86 s	0.021	0.011

Least-loaded edges p2c on p99 at both sizes, as expected here: one router, so the herd p2c exists to break up never forms. p2c takes imbalance at 8 workers (0.011 vs 0.021). The herd does appear in T3. plots/soak.png: random drifts away at ~55 s and never recovers.

Part 3, T2 — does prefix-aware routing save KV?

traces/t2_shared_prefix.jsonl — 959 requests, 100 s at 10 req/s, 384 sharing one prefix, DRR, 2 workers. KV block-seconds = blocks held, integrated over time; peak alone

cannot separate a strategy that holds the same blocks twice as long.

Framing	Baseline	Treatment	KV block-s	ratio	also	verdict
A — as the assignment words it	least_loaded, <i>cache off</i>	prefix_then_load, cache on	266,380 → 92,038	0.346	peak blocks 9,343 → 2,145 (0.230)	clears the 0.40 target
B — same mechanism both sides	least_loaded, <i>cache on</i>	prefix_then_load, cache on	115,931 → 92,038	0.794	prefill tokens ratio 0.851	target unreachable

The answer depends entirely on the baseline. A measures cache and routing together against a system with neither and clears 0.40. B gives the baseline the same cache to isolate the *routing* contribution, and 0.40 is then mathematically out of reach: two workers each holding a copy of the prefix, collapsed to one, has a floor of 0.5 on that component, and the prefix is only part of the total. 0.794 against a 0.5 floor on part of the term is routing working, not failing. An analytic check on the trace's own token counts agrees with A: 1,989,772 KV tokens prefix-unaware against 649,272 prefix-aware, ratio 0.326.

Prefix affinity with no cache to pay it back is actively harmful. prefix_then_load with the cache off cost 543,588 block-seconds against least-loaded's 266,380 — **2.0× worse** — p99 8.45 s against 0.69 s, imbalance 0.243 against 0.016, 125 preemptions against zero. Affinity concentrates traffic by design; buy no prefill saving with it and all that is left is the imbalance.

Part 3, T3 — stale telemetry, and what H6 actually costs

traces/t3_stale.jsonl — 578 requests, 120 s at 4.5 req/s, DRR, least_loaded; worker w1's load scrape arrives 15 s late. **stale_hidden** reports that view as fresh so H6 cannot see it; **stale_honest** reports the age truthfully, so H6 classes w1 unknown. H6 is unconditional inside router.py, so the experiment lies to it rather than disabling it.

Arm	2w p99 TTFT	2w shed	2w completed	2w KV block-s	8w p99 TTFT	8w shed	8w completed
fresh (control)	1.21 s	0%	536	671,979	0.15 s	0%	2,022
stale_hidden (no H6)	8.17 s	0%	531	749,890	65.24 s	15.6%	1,559
stale_honest (H6 fires)	23.35 s	29.4%	320	796,912	0.82 s	0%	2,018

The two-worker result contradicts what the exercise expects, and it is the more interesting half. At N=2 the H6 guard makes the outcome far worse than the failure it prevents. w1's telemetry is 15 s old for the whole run, so H6 classes it unknown for the whole run, takes half the fleet offline and lands every placement on w0: p99 23.35 s and 29.4% shed, against 8.17 s and zero shed for the same lag with H6 blinded. At N=8 the same guard is clearly right — 0.82 s and no shedding, against 65.24 s and 15.6% shed without it. **H6 costs one worker in N.** At N=8 that is 12.5% of capacity to avoid a herd onto a machine whose queue is invisible; at N=2 it is half the fleet to avoid a problem worth 8 s. The rule is right; applying it unconditionally to a two-machine fleet is not. The fix is a floor on surviving capacity rather than a blanket exclusion — the shape of the all-unknown fallback already in _eligible.

The one place p2c earns its keep: at 8 workers with staleness hidden it holds p99 to 9.30 s where least_loaded reaches 65.24 s. That gap *is* the herd. plots/t3_stale.png plots the 29.4% rejection trace.

The eight questions — where each one is answered in the code

Question	Where it is answered
Where do I prevent accepting work that will time out?	admit_check_queue_wait (gate 3) — expected queue wait vs half of req.deadline_s
Where do I protect KV memory?	admit_check_kv_pressure and _check_kv_capacity before entry; sched_kv_room and _reclaim_blocks after
Where do I prioritise interactive traffic?	admit_check_tail_latency (gate 6) sheds batch under a blown tail; sched_select_priority orders the GPU; _select_victim refuses to evict upward
Where do I stop one tenant monopolising?	admit_check_allowance (gates 1–2, 429) before entry; sched_select_drr shares the prefill slot after
Where do I preempt a request?	sched_preempt, triggered by _reclaim_blocks, victim chosen by _select_victim
Where do I exploit shared prefixes?	router_prefix_then_load picks the warm worker; _cached_tokens feeds admission gate 4
Where do I avoid sending work to a worker that has gone quiet?	router_eligible / _is_unknown (H6), with _load_key sorting an unknown view as maximally loaded behind it
Where do I avoid bouncing a request around the fleet?	router_admissible / _would_shed (H4) — predicts the refusal before dispatch, returns one Shed(503, retry_after=2.0)

Prefixes above: admit. is gateway/admit.py, sched. is gateway/sched.py, router. is router.py. **Known gaps.** Nothing reserves capacity, so two requests inside one scrape interval can both be admitted against the same free blocks. retry_after is a placeholder (1.0 s / 2.0 s). Priority is trusted as sent. BLOCK_SIZE = 16 exists in four places, now pinned by tests/test_constants_agree.py rather than by comment. 320 tests pass; make part3 reproduces every number here.